1. **Problem Statement**
2. **User Stories**
3. **High Level Design**
4. **Class Diagram**
5. **Low Level Design**

Project Title: Payments Management System

Problem Statement: A fintech startup requires a simple, secure, and efficient internal Payments Management System to accurately track all financial transactions. The system must support real-time updates, ensure data integrity, and provide traceability for audit purposes. As part of the development team, you are tasked with designing and implementing this system as a Java console-based application suitable for internal use.

User Stories – Payments Management System

User Story 1 – Admin Login

Title: Admin Authentication

As an Admin **I want to** securely log into the system **So that** I can manage payments, users, and audit logs.

Acceptance Criteria:

- Admin must provide valid username and password.
- Role must be ADMIN.
- Failed authentication should deny access.
- Successful login should be logged in audit logs.

User Story 2 – Create Finance Manager

As an Admin **I want to** create Finance Manager accounts **So that** they can record payment transactions.

Acceptance Criteria:

- Only Admin can create Finance Managers.
- Username must be unique.
- Creation must be recorded in audit logs.

User Story 3 – Add Payment (Finance Manager)

As a Finance Manager
I want to record credit and debit payments
So that company financial transactions are tracked.

Acceptance Criteria:

- Payment must have unique ID.
- Payment status defaults to PENDING.
- Payment must store creator username.
- Audit log must record the operation.

User Story 4 – Update Payment Status (Admin)

As an Admin
I want to approve or reject payments
So that transactions are properly validated.

Acceptance Criteria:

- Only Admin can update status.
- Payment must exist.
- Updated status must be logged.
- Updated timestamp and user must be recorded.

User Story 5 – Concurrent Payment Recording

Title: Simultaneous Payment Creation

As multiple Finance Managers **We want to** record payments at the same time **So that** high transaction volumes can be handled efficiently.

Concurrency Risk Identified:

- Duplicate payment IDs
- Lost updates
- Corrupted JSON file
- Inconsistent payment count

User Story 6 – Concurrent Payment Status Update

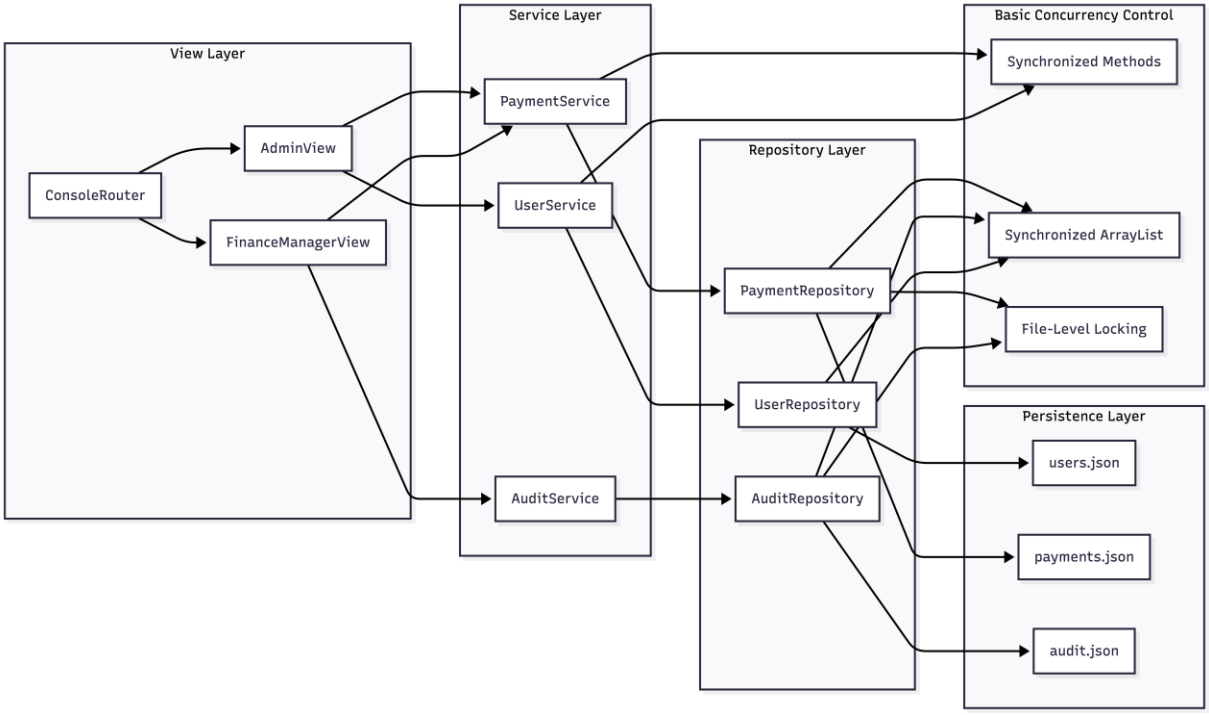
As multiple Admin users
We want to update the status of the same payment concurrently
So that the system handles simultaneous decision-making correctly.

Concurrency Risk Identified:

- Race condition
- Last-write-wins problem
- Inconsistent audit logs
- Data overwrite

Solution Implemented:

- Synchronized repository update
- Proper logging of each thread action



High-Level Design (HLD)

1. Architectural Overview

The Payments Management System (PMS) follows a layered architecture to ensure separation of concerns, maintainability, and structured interaction between components. The system is divided into five major logical sections:

1. View Layer
2. Service Layer
3. Repository Layer
4. Basic Concurrency Control
5. Persistence Layer

Each layer has clearly defined responsibilities and interacts only with adjacent layers to maintain modularity and clean architecture.

2. View Layer

Components:

- ConsoleRouter
- AdminView
- FinanceManagerView

Responsibilities:

The View Layer handles all user interactions through the console interface. It does not contain business logic. Instead, it collects user input, displays outputs, and delegates requests to the appropriate services.

- ConsoleRouter manages authentication flow and routes users to role-specific dashboards.
- AdminView allows the Admin to create finance managers, view payments, update payment status, and view audit logs.
- FinanceManagerView allows Finance Managers to add payments and view their transactions.

The View Layer communicates directly with the Service Layer.

3. Service Layer

Components:

- PaymentService
- UserService
- AuditService

Responsibilities:

The Service Layer contains the core business logic of the system. It acts as a bridge between the View Layer and the Repository Layer.

- PaymentService handles payment creation, status updates, and retrieval.
- UserService manages authentication and user creation.
- AuditService records and retrieves audit logs.

This layer ensures that business rules are enforced before any data is persisted or retrieved.

The Service Layer interacts with both the Repository Layer and the Basic Concurrency Control mechanisms.

4. Repository Layer

Components:

- PaymentRepository
- UserRepository
- AuditRepository

Responsibilities:

The Repository Layer manages data storage and retrieval operations. It maintains in-memory collections and handles JSON file persistence.

- PaymentRepository manages payment records.
- UserRepository manages user accounts.
- AuditRepository manages audit logs.

This layer isolates data access logic from business logic, ensuring a clean separation between application behavior and data management.

5. Basic Concurrency Control

Mechanisms Implemented:

- Synchronized Methods
- Synchronized ArrayList
- File-Level Locking

Purpose:

Since multiple operations may attempt to access shared resources simultaneously (such as payment lists or JSON files), basic concurrency control mechanisms are implemented.

1. **Synchronized Methods**
   Critical methods in service and repository layers are synchronized to prevent race conditions during data modification.
2. **Synchronized ArrayList**
   In-memory collections storing users, payments, and audit logs are protected using synchronized lists to prevent concurrent modification issues.

3. **File-Level Locking**
   JSON file write operations are controlled to ensure that only one write operation occurs at a time, preventing file corruption.

These mechanisms ensure data consistency, integrity, and safe multi-user operation.

6. Persistence Layer

Data Files:

- users.json
- payments.json
- audit.json

Responsibilities:

The Persistence Layer is responsible for permanent data storage. All data from the repository layer is serialized into JSON format and stored in respective files.

This ensures:

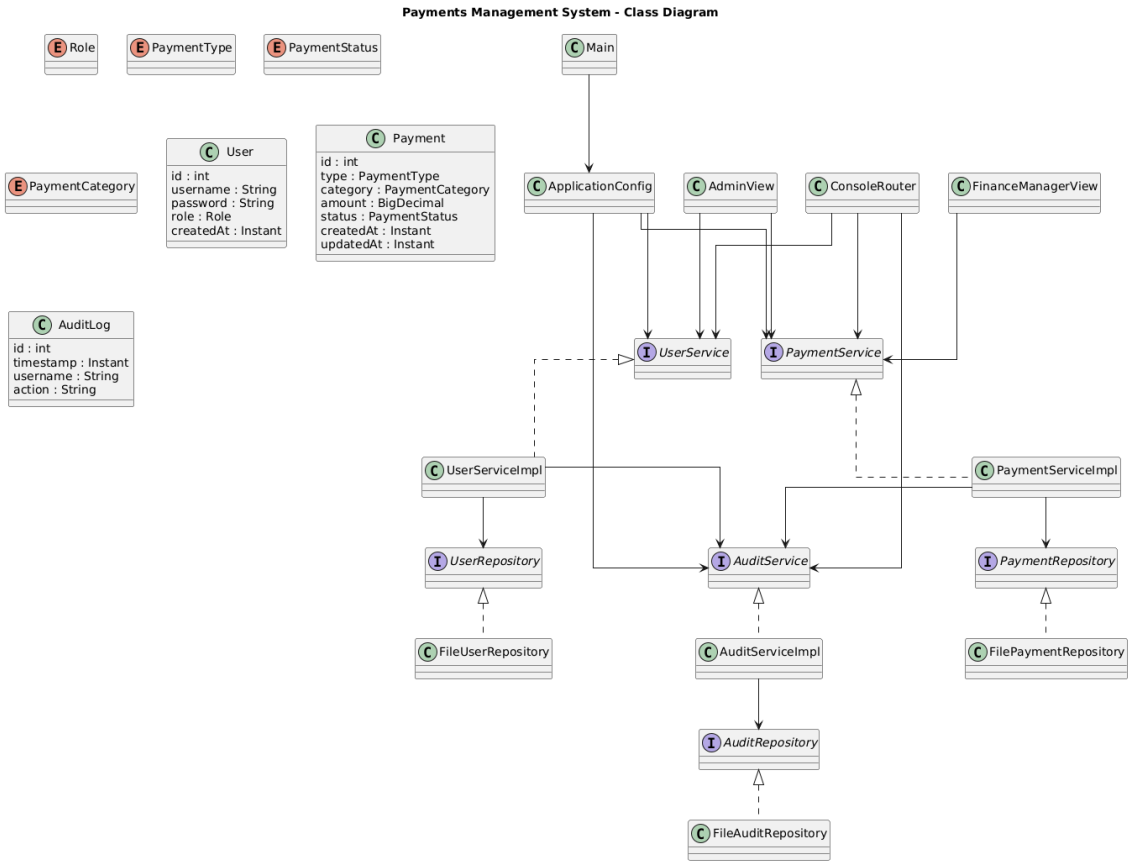
- Data persistence across application restarts
- Structured storage
- Separation of application logic from storage format

7. Interaction Flow Summary

The overall system interaction follows this sequence:

1. The user interacts with the View Layer.
2. The View Layer forwards requests to the Service Layer.
3. The Service Layer applies business rules.
4. The Repository Layer performs data operations.
5. Concurrency controls ensure safe access to shared resources.
6. Updated data is persisted in JSON files.
7. Audit logs record system activity.

Class Diagram Description



1. Overview

The class diagram represents the structural design of the Payments Management System. It illustrates the relationships between entities, service interfaces, repository interfaces, their implementations, and the view layer components. The system follows a layered architecture with clear separation between presentation, business logic, and data access layers.

The diagram includes:

- Entity classes
- Enumerations
- Service interfaces and implementations
- Repository interfaces and implementations
- View layer components
- Configuration and main entry point

2. Enumerations

The system defines the following enumerations to enforce controlled value sets:

2.1 Role

Defines user roles:

- ADMIN
- FINANCE_MANAGER

2.2 PaymentType

Defines payment nature:

- CREDIT
- DEBIT

2.3 PaymentStatus

Defines transaction lifecycle:

- PENDING
- PROCESSING
- COMPLETED

2.4 PaymentCategory

Defines payment classification:

- SALARY
- VENDOR_PAYMENT
- CLIENT_PAYMENT

These enumerations ensure type safety and prevent invalid values.

3. Entity Classes

3.1 User

Represents system users.

Attributes:

- id : int
- username : String
- password : String
- role : Role

- createdAt : Instant

A User can either be an Admin or Finance Manager.

3.2 Payment

Represents a financial transaction.

Attributes:

- id : int
- type : PaymentType
- category : PaymentCategory
- amount : BigDecimal
- status : PaymentStatus
- createdAt : Instant
- updatedAt : Instant

This class models the core business entity of the system.

3.3 AuditLog

Represents system activity logs.

Attributes:

- id : int
- timestamp : Instant
- username : String
- action : String

AuditLog ensures traceability of system operations.

4. View Layer Classes

4.1 ConsoleRouter

Handles authentication flow and routes users to appropriate views.

4.2 AdminView

Provides admin-specific functionalities:

- Create Finance Manager
- Update payment status
- View payments
- View audit logs

4.3 FinanceManagerView

Provides finance manager functionalities:

- Add payment
- View personal payments

The view layer depends on service interfaces only, ensuring loose coupling.

5. Service Layer

The service layer defines business logic through interfaces and implementations.

5.1 Service Interfaces

- UserService
- PaymentService
- AuditService

These interfaces define system operations without exposing implementation details.

5.2 Service Implementations

- UserServiceImpl
- PaymentServiceImpl
- AuditServiceImpl

Each implementation class realizes its corresponding interface and contains business logic.

Relationships:

- UserServiceImpl depends on UserRepository and AuditService
- PaymentServiceImpl depends on PaymentRepository and AuditService

- AuditServiceImpl depends on AuditRepository

This design promotes abstraction and maintainability.

6. Repository Layer

The repository layer handles data persistence.

6.1 Repository Interfaces

- UserRepository
- PaymentRepository
- AuditRepository

These define data access methods.

6.2 Repository Implementations

- FileUserRepository
- FilePaymentRepository
- FileAuditRepository

These classes implement their respective interfaces and manage JSON file-based persistence.

Relationships:

- FileUserRepository implements UserRepository
- FilePaymentRepository implements PaymentRepository
- FileAuditRepository implements AuditRepository

This abstraction allows future migration to database storage without affecting business logic.

7. Configuration and Entry Point

7.1 ApplicationConfig

Acts as a dependency injector.
It initializes repositories, services, and wires dependencies together.

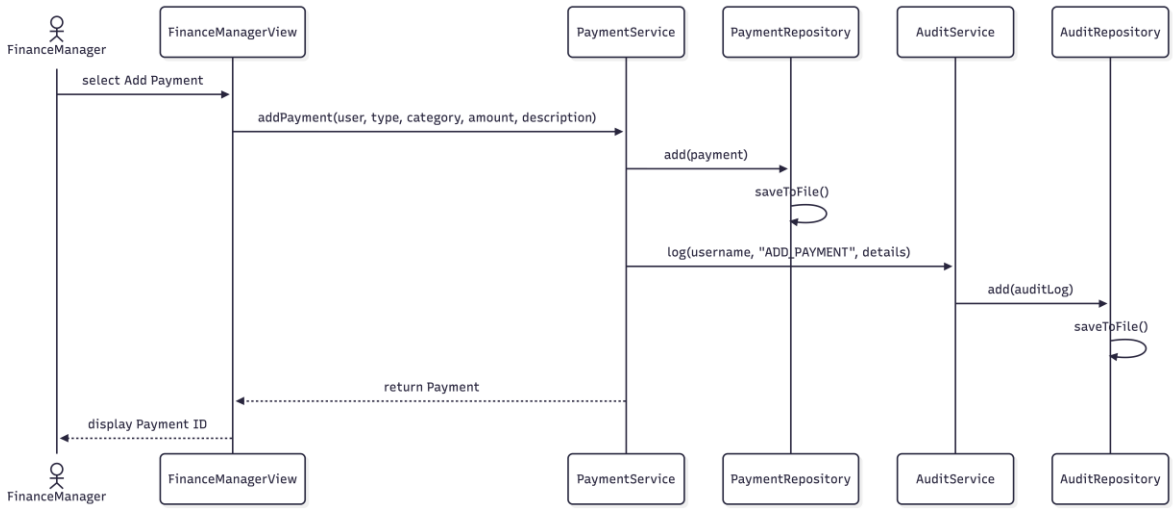
7.2 Main

The entry point of the application.
It creates the ApplicationConfig instance and starts the ConsoleRouter.

8. Relationship Summary

The diagram reflects the following architectural principles:

1. Interface-based design
   Services and repositories are defined using interfaces and implemented separately.
2. Dependency direction
   View → Service → Repository → Persistence
3. Loose coupling
   Higher-level modules depend on abstractions, not concrete implementations.
4. Layered separation
   Each layer has a clearly defined responsibility.



5.1 LLD – Add Payment Flow (Finance Manager)

Overview

This sequence diagram illustrates the internal interaction flow when a Finance Manager adds a new payment transaction into the system.

Step-by-Step Flow Explanation

1. The Finance Manager selects the “Add Payment” option from the FinanceManagerView.
2. The FinanceManagerView collects input details such as:
    a. Payment type (Credit/Debit)
    b. Category
    c. Amount
    d. Description
3. The view layer calls:

```
PaymentService.addPayment(user, type, category, amount, description)
```

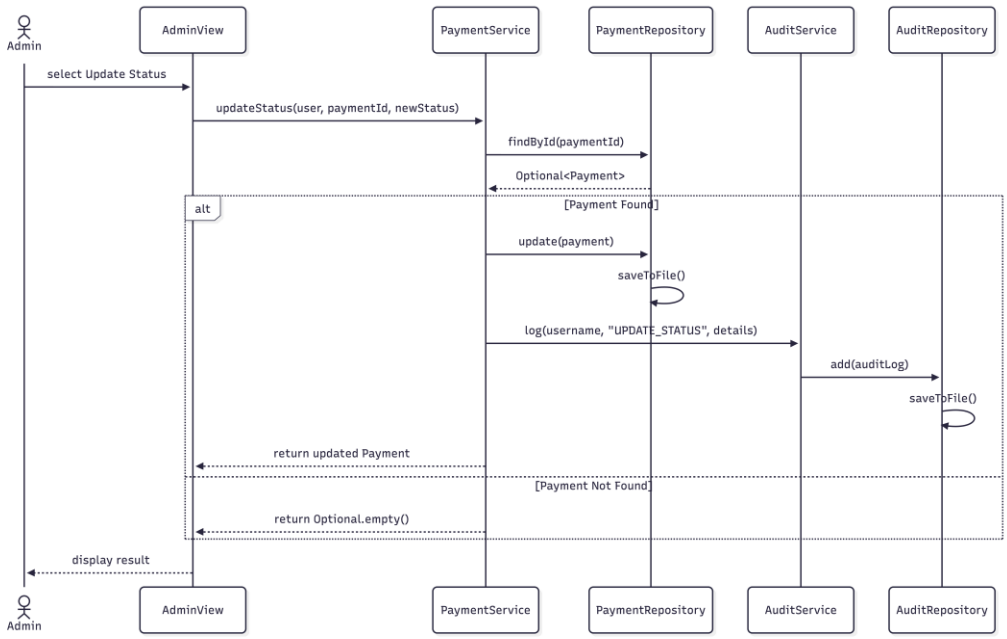
4. Inside PaymentService:
    a. A unique payment ID is generated.
    b. A new Payment object is created.
    c. Default status is set to PENDING.
    d. createdAt and updatedAt timestamps are assigned.
5. PaymentService calls:

```
PaymentRepository.add(payment)
```

6. The repository:
    a. Adds the payment to the in-memory list.
    b. Persists the updated list into payments.json using saveToFile().
7. After successful persistence, PaymentService calls:

```
AuditService.log(username, "ADD_PAYMENT", details)
```

8. AuditService:
    a. Creates a new AuditLog object.
    b. Calls AuditRepository.add(log).
    c. The repository persists the log into audit.json.
9. The Payment object is returned back to FinanceManagerView.
10. The payment ID is displayed to the user.



5.2 LLD – Update Payment Status Flow (Admin)

Overview

This sequence diagram represents the internal processing when an Admin updates the status of an existing payment.

Step-by-Step Flow Explanation

1. The Admin selects “Update Payment Status” from AdminView.
2. AdminView calls:

PaymentService.updateStatus(user, paymentId, newStatus)

3. PaymentService requests:

PaymentRepository.findById(paymentId)

4. Repository searches the in-memory list:
    a. If not found → returns Optional.empty().
    b. If found → returns the Payment object.

Case 1: Payment Found

5. PaymentService updates:
    a. status
    b. updatedAt timestamp
    c. updatedBy username
6. PaymentService calls:

PaymentRepository.update(payment)

7. Repository replaces the existing object and saves to payments.json.
8. PaymentService logs the update using:

AuditService.log(...)

9. AuditRepository persists the log into audit.json.
10. Updated Payment is returned to AdminView.

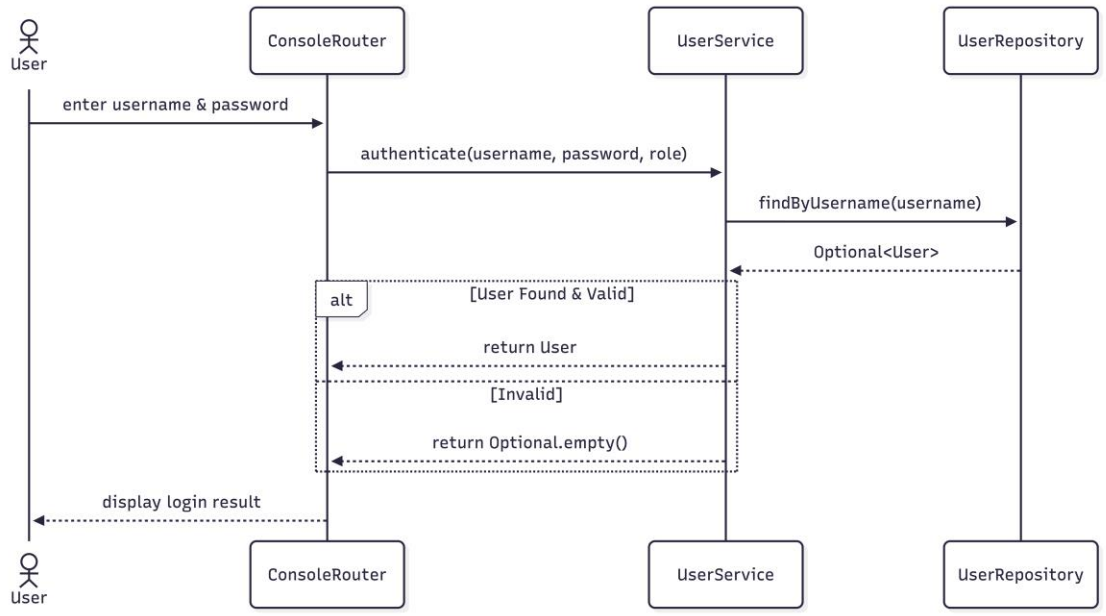
Case 2: Payment Not Found

- Optional.empty() is returned.
- AdminView displays an error message.

Business Rules Enforced

- Only Admin can update status.
- Payment must exist.
- Every status change must be logged.
- updatedAt timestamp must be refreshed.

5.3 LLD – Authentication Flow



Overview

This sequence diagram describes the authentication mechanism for both Admin and Finance Manager users.

Step-by-Step Flow Explanation

1. User enters username and password in ConsoleRouter.
2. ConsoleRouter calls:

UserService.authenticate(username, password, role)

3. UserService calls:

UserRepository.findByUsername(username)

4. Repository searches for the user in memory.

Case 1: User Found and Valid

5. UserService verifies:
    a. Password correctness.
    b. Role matches requiredRole.
6. If valid:
    a. Returns Optional.of(user).

b. Login success message is shown.
c. Audit log entry is created for login.

Case 2: Invalid Credentials

- If username not found → Optional.empty().
- If password mismatch → Optional.empty().
- If role mismatch → Optional.empty().
- Login failed message is displayed.

Business Rules Enforced

- Username must exist.
- Password must match stored password.
- Role must match selected login type.
- Failed attempts are logged.

5.4 Concurrency Considerations in LLD

Although the system is console-based, shared resources are protected to prevent inconsistencies.

Protected Resources:

- Payment list
- User list
- Audit log list
- JSON files

Mechanisms Used:

- Synchronized methods in repository layer
- Synchronized collections
- Controlled file write operations

This ensures:

- No race conditions
- No data corruption
- No inconsistent updates
- Safe concurrent access